Tokenization and Stemming Approaches for Turkish

Tarık Emre Tıraş
Department of Linguistics
Boğaziçi University
Istanbul, Turkey
tarik.tiras@boun.edu.tr

Abstract

This paper presents three basic preprocessing tools for Turkish: a rule-based stemmer, a rule-based tokenizer, and a simple machine-learning tokenizer. These tools form part of a larger preprocessing pipeline, but here we only focus on the three components we implemented. The aim is to offer practical and UD-compatible methods that work on general-domain Turkish texts. The stemmer uses an ordered list of suffix rules, several repair steps for morphophonemic changes, and a lexicon of about 38K root forms. When tested on the BOUN-UD Turkish Treebank, it reaches 80.2% strict accuracy and 93.3% soft accuracy. The strict score is lower because UD often assigns lemmas at a very fine level, especially for derivational forms. The rule-based tokenizer relies on regular expressions for URLs, e-mails, hashtags, punctuation, and numbers. It achieves 99.73% sentence-level exact match and a 0.999 average Jaccard similarity with the UD gold tokens. There is also an optional MWE mode. Since this part was tested on a small annotated subset, its score (0.82 F1) mainly reflects the difficulty of detecting MWEs in Turkish. The logistic-regression tokenizer uses manually designed character-level features and offers a simple statistical alternative. It reaches about 0.99 accuracy and 0.97 macro F1 on the UD test set. Overall, these results show that rule-based methods and lightweight models can still provide strong baselines for Turkish preprocessing.

1 Introduction

Turkish is a morphologically rich and mostly agglutinative language. Because of its productive suffixation and flexible word formation, downstream NLP tasks are often sensitive to segmentation and morphological simplification. Tools such as tokenizers and stemmers play a central role in preparing consistent input for parsers and other sequence-based models. For practical purposes, these tools must also be compatible with established annotation schemes, including Universal Dependencies (UD) (Nivre et al. 2016).

Several resources exist for Turkish, including the BOUN-UD Treebank (Türk et al. 2020), the Kenet UD treebank, and Turkish WordNet (Erjavec, Ljubešić, Scherrer, Sulubacak, et al. 2022a; Erjavec, Ljubešić, Scherrer, Sulubacak, et al. 2022b; Erjavec, Ljubešić, Scherrer, and Tyers 2022). Widely used systems such as Zemberek (A. A. Akın and M. D. Akın 2007; A. Akın and M. Akın 2007) and the Snowball Turkish stemmer (Porter 2001) provide strong baselines, but their internal rules are not always easy to inspect or adapt. Tokenization also poses practical challenges, especially for handling clitics, apostrophe suffixes, abbreviations, and multiword expressions (MWEs) (Eryiğit et al. 2015).

This paper presents a set of transparent, controllable components for Turkish stemming and tokenization. The main contributions are:

- A rule-based stemmer built from a hand-compiled suffix inventory, a lexicon of roughly 38K root forms, and a small set of morphophonemic repair rules.
- A rule-based tokenizer that uses Turkish-specific regular expressions, abbreviation and apostrophe handling, and optional MWE merging based on a lexicon derived from Turkish WordNet.
- A character-level logistic regression tokenizer with manually designed features and an incremental ablation setup on UD data.

- A unified evaluation of all components with respect to UD-style lemmas, tokens, and MWEs.

Section 2 gives background on Turkish morphology and tokenization. Section 3 describes the stemmer. Section 4 explains the rule-based tokenizer and its MWE handling. Section 5 presents the logistic regression tokenizer. Section 6 discusses errors and limitations, and Section 7 concludes.

2 Background

Turkish is a mainly agglutinative language, and surface forms often contain long sequences of inflectional and derivational suffixes. Because of this structure, many stemmers rely on ordered suffix stripping supported by a detailed list of suffixes and a reasonably broad lexicon of root forms. Existing work ranges from rule-based tools such as Snowball (Porter 2001) to full morphological analyzers like Zemberek (A. A. Akın and M. D. Akın 2007; A. Akın and M. Akın 2007). More recent studies also explore graph-based approaches that use morphological similarity (Arslan and Orhan 2016). These studies note that strict lemmatization accuracy may remain limited even for strong analyzers, while softer and ambiguity-aware metrics usually give a more realistic view of morphological coverage.

Tokenization for Turkish is shaped by the segmentation principles of Universal Dependencies (UD) (Universal Dependencies Consortium 2023). UD treebanks such as BOUN (Türk et al. 2020) and Kenet (Erjavec, Ljubešić, Scherrer, Sulubacak, et al. 2022a; Erjavec, Ljubešić, Scherrer, Sulubacak, et al. 2022b) apply a relatively conservative policy: they separate punctuation and clitics but generally keep multiword expressions (MWEs) unmerged. At the same time, MWEs often behave as single lexical units. Prior work on Turkish MWE extraction (Eryiğit et al. 2015) reports F1 scores around 0.60–0.63, showing both the difficulty of the task and the potential value of limited, controlled MWE handling. These observations suggest that a tokenizer should preserve UD compatibility while still allowing optional merging of clearly lexicalized MWEs.

3 Rule-based Turkish Stemmer with Suffix Lists and Repairs

3.1 Architecture

The stemmer follows a simple pipeline. Each surface token is first normalized, then passed through ordered suffix stripping and a small set of repair rules. Candidate stems are checked against a root lexicon, and the system returns a single stem for each token. For evaluation, predicted stems are compared with UD lemmas on a held-out development set.

3.2 Suffix Inventory and Lexicon

We created two specific suffix lists manually, based on standard descriptions of Turkish morphology:

- **Inflectional suffixes** (plural, case, tense, agreement),
- **Derivational suffixes** (nominalizers, verbalizers, negation, causatives).

Before use, both lists are cleaned by removing duplicates and sorting suffixes from longest to shortest, so that more specific forms are matched first. The infinitival endings *-mak/-mek* are handled early in the derivational list to avoid conflict with shorter, similar-looking suffixes.

The lexicon is crucial for deciding which candidates are acceptable roots. It is created by combining lemmas from Zemberek and the BOUN-UD treebank, followed by deduplication and normalization. The final list contains about 38K items and is stored as a plain text file. A short list of exceptions is added to handle irregular verbs and other cases where rule-based stripping alone is insufficient.

3.3 Unicode Normalization

During development, inconsistencies in dotted and undotted *i* characters caused many mismatches. Tokens that look identical may differ in their Unicode composition. NFKC and NFC are standard Unicode normalization forms that convert visually similar characters into a consistent underlying representation. To avoid mismatches, all tokens are passed through a shared normalization step (NFKC/NFC), followed by Turkish-specific lowercasing and a small set of extra replacements to handle uncommon dotted-*i* variants. This ensures that predicted stems and gold lemmas are compared under the same representation.

3.4 Stripping Strategy and Repairs

Given a normalized token, the stemmer performs a small number of ordered operations:

1. **Inflectional stripping:** try removing one inflectional suffix, preferring longer matches and keeping a minimum stem length.
2. **Derivational stripping:** repeat the process for derivational suffixes.
3. **Vowel repairs:** if the result is not a valid root, apply a few common vowel alternation mappings to undo frequent morphophonemic changes.
4. **Final-consonant repairs:** if needed, restore final consonants that undergo devoicing (*b*→*p*, *d*→*t*, .g→*k*) and handle simple gemination (e.g., *tt*→*t*).
5. **Secondary derivation check:** apply one more derivational stripping attempt followed by a light repair step to support stacked derivations.
6. **Exception list:** if no candidate becomes a root, try a few manually curated irregular mappings (e.g. *edildi*→*et*, *yapıldı*→*yap*).

A candidate is accepted as soon as it appears in the lexicon. If all attempts fail, the original token is returned unchanged. A few targeted heuristics are included, for example limiting the stripping of *ma/me* to avoid over-removing deverbal nouns. For ambiguous forms ending in *t* or *d*, the stemmer prefers the form that becomes a valid root when either consonant is restored.

3.5 Evaluation and Soft Accuracy

We evaluate the stemmer on the development portion of the BOUN-UD Turkish Treebank, using token–lemma pairs as gold labels and ignoring AUX tokens. Two metrics are reported:

- **Strict accuracy:** exact match between predicted stems and gold lemmas.
- **Soft accuracy:** similarity-based scoring using Python's `difflib.SequenceMatcher`. A prediction is counted as correct if the similarity is at least 0.7.

The stemmer reaches 80.2% strict accuracy and 93.3% soft accuracy. The difference reflects that many "errors" are close variants of the gold lemma. For example, *bekliyordu*→*bekli* and *yazılmış*→*yazı* fail strict scoring but remain near the true form. Similar trends have been reported elsewhere; for instance, (Arslan and Orhan 2016) report relatively low exact-match scores for Zemberek but much higher ambiguity-aware accuracy for their graph-based lemmatizer.

4 Rule-based Tokenizer with Optional MWE Merging

4.1 System Overview

The tokenizer is designed to be UD-compatible while offering flexibility for MWEs. It processes text in stages:

- a regular-expression layer for URLs, e-mails, hashtags, numbers, time expressions and punctuation;
- post-processing for abbreviations, Roman numerals and apostrophe suffixes;
- an optional MWE-merging step based on a lexicon extracted from Kenet Turkish WordNet (Erjavec, Ljubešić, Scherrer, and Tyers 2022).

The tokenizer runs in two modes: (i) UD-compatible (no MWE merging), and (ii) MWE-aware (MWEs merged into single tokens).

4.2 Regex Layer and Initial Segmentation

The first layer applies a compiled pattern of Turkish-specific regular expressions. Specific patterns (decimals, clock times) are prioritized over broader ones to prevent incorrect splits. This step separates punctuation and preserves web entities (URLs, hashtags) and numeric formats.

4.3 Abbreviations, Roman Numerals and Apostrophe Suffixes

A second layer adjusts the initial segmentation in three ways:

- **Abbreviations:** tokens ending with a period are checked against a manually expanded abbreviation list (adapted from Zemberek but simplified for our use). True abbreviations stay as single tokens; others are split into a word and a following period.
- **Roman numerals:** forms like *I.* or *II.* are detected and kept as single units, which avoids incorrect splitting of chapter or section markers.
- **Apostrophe suffixes:** forms such as *Ankara ' ya* are merged into *Ankara'ya* using a small regular-expression pattern and a list of valid suffixes. This follows standard orthographic practice for proper names in Turkish.

These adjustments make the output closer to UD boundaries while respecting common orthographic usage.

4.4 MWE Lexicon

For MWE-aware tokenization, we build a lexicon of multiword expressions (MWEs) from the Turkish WordNet (KENET). All XML files are scanned for `<LITERAL>` elements whose surface forms contain a space. This produces multiword lemmas such as *mor çiçek*, *kara gün*, *kuru fasulye*, and many multiword proper nouns. Each entry is Unicode-normalized and lowercased with a Turkish-specific function. The resulting list contains several thousand MWEs and serves as the only MWE source for the system. Since the lexicon comes directly from WordNet, it mainly captures lexicalized expressions and may miss idioms that are not listed in the XML files.

4.5 MWE Matching and Merging

After regex-based segmentation, the tokenizer searches for MWEs using a longest-match approach. For each token index i, spans of length L down to 2 (where L is the maximum MWE length) are checked. Two matching strategies are used:

Exact matching. A span is merged if its lowercased surface form appears in the lexicon (e.g., "kara kedi" $\rightarrow$ "kara_kedi"). This handles MWEs without inflection on the head.

Head-lemma matching. For MWEs of length ≥ 2, the lexicon also stores the base (all but the last token) and possible head lemmas. A span is merged when the base matches and the final token begins with a head lemma, allowing inflected variants (for example "ense köküme" $\rightarrow$ "ense_köküme").

Representation. Merged MWEs are written as single underscore-joined tokens (e.g., "kuru fasulye" $\rightarrow$ "kuru_fasulye").

4.6 Evaluation on an Independent Gold Standard

To evaluate MWE merging independently of UD tokenization, we use a manually prepared gold standard of 201 sentences annotated with merged MWEs. This dataset is not used by the tokenizer during processing; it is only for evaluation. It includes idiomatic MWEs, multiword proper nouns, and expressions that depend on context.

During scoring, both predicted and gold MWEs are identified by the presence of an underscore. Standard precision, recall and F1 are computed. The results are shown below:

	Precision	Recall	F1
MWE Merger	0.7042	0.9883	0.8224

Most false positives come from the head-lemma heuristic, which can over-generate when an inflected noun accidentally begins with the same substring as a WordNet lemma. False negatives mostly correspond to MWEs missing from the WordNet-derived lexicon, so the system simply cannot detect them.

4.7 Evaluation

UD-compatible mode. In UD-compatible mode, MWE merging is disabled so that the output can be compared directly with UD boundaries. Evaluation is carried out on a Turkish UD treebank (Kenet) using two metrics:

- **Sentence-level exact match**: a sentence is correct if the token list matches the UD list exactly.
- **Token Jaccard similarity**: for each sentence, we compute the Jaccard similarity between the predicted token forms and the gold forms, and then average the similarity.

In this mode, the tokenizer reaches 99.73% sentence-level exact match and an average token Jaccard similarity of 0.999.

MWE-aware mode. In MWE-aware mode, the tokenizer is evaluated on the same manually annotated dataset of 201 sentences, where target MWEs appear as single tokens (e.g., *ense köküme*). The dataset is independent of the lexicon to avoid circularity.

Using standard MWE metrics, the tokenizer obtains precision $= 0.70$, recall $= 0.99$, and F1 $= 0.82$ (TP $= 169$, FP $= 71$, FN $= 2$). Error analysis shows that many false positives involve expressions that are sometimes idiomatic and sometimes compositional, which makes perfect filtering difficult.

Combining both structural and lexical evaluations gives an overall tokenization quality of roughly 91% when averaging the near-perfect structural score and the MWE F1 score. This is broadly consistent with prior reports on Turkish MWE extraction (Eryiğit et al. 2015), although direct comparison is difficult due to different datasets.

5 Character-level Logistic Regression Tokenizer

5.1 Data and Labeling

The tokenizer works at the character level. Data are taken from the train, development and test splits of the BOUN-UD corpus (Türk et al. 2020). For each sentence, the raw string is read from the UD `# text` comment, and token boundaries are recovered from the accompanying token list.

Each character position is labeled as:

- **1**: start-of-token, if it matches the first character of a UD token;
- **0**: otherwise.

Token start indices are found by scanning the raw text from left to right and matching each token surface form from the point where the previous match ended. This avoids re-matching earlier occurrences. Sentences that do not include a `# text` line are skipped. Using this method, the training split contains roughly 608K labeled characters, with the development and test sets processed in the same way.

5.2 Feature Design

Each character is represented by a simple binary feature vector. The baseline model uses 15 features that encode the character type (uppercase, lowercase, digit, whitespace, punctuation), the types of its immediate neighbors, and basic BOS/EOS markers. On top of this baseline, three additional feature groups are defined:

- **Group 1:** finer alphanumeric-type flags that refine the coarse character categories;
- **Group 2:** small indicators for characters such as `@`, `#`, `/`, `:`, and `.`, which are common in URLs, e-mail addresses and numeric patterns;
- **Group 3:** a few simple pattern-based cues, such as digit–punctuation–digit sequences (often marking decimals or time expressions), period-before-uppercase (common in abbreviations), and basic URL/e-mail shape cues.

We evaluate five settings: the 15-feature baseline; baseline plus Group 1, Group 2 or Group 3; and a combined 27-feature model. This provides a straightforward ablation over increasingly rich feature sets.

5.3 Model Training and Ablation

A logistic regression classifier with ℓ_2 regularization predicts the boundary label for each character. The main model is trained on the training split, with the development split used for tuning. Final scores are reported on the BOUN-UD test split.

To see how individual feature groups affect performance, all five configurations are trained and evaluated in the same way. For quicker experiments, some of the ablation runs use the development split as

both training and validation data, while keeping the test split untouched. In practice, all models perform well, and the full 27-feature model gives the best results.

5.4 Results

The baseline model already gives strong performance, showing that many token boundaries can be found from local character cues alone. Adding Groups 1 and 2 produces small but steady improvements, especially for numerical and web-like tokens. Group 3 helps with more difficult cases involving decimals, times and abbreviation-like patterns.

The final model with all feature groups achieves:

- about 0.99 character-level accuracy;
- about 0.97 macro F1 across the two classes (boundary vs. continuation).

These results suggest that fairly simple character-level features, together with a linear classifier, are enough to approximate UD-style token boundaries for Turkish with high accuracy. The model does not handle MWE merging on its own, but it provides a clean, language-specific baseline that can be combined with lexicon-based post-processing if needed.

6 Discussion

The experiments show several trade-offs and limitations in stemming and tokenization for Turkish. For stemming, one of the main challenges is the overlap between different morphophonemic variants and suffix classes. Without POS information or a full morphological analysis, it is often unclear whether a surface form reflects an inflectional or derivational ending. The shallow stripping strategy, together with lexicon checks and a few targeted heuristics (such as those for *ma/me* and final *t/d*), prevents many over-stripping errors, although some derived forms still remain problematic. Soft accuracy helps capture these "almost correct" cases by giving partial credit.

For tokenization, the UD-compatible mode is relatively predictable: once common patterns and exceptions are covered, matching UD boundaries becomes mostly a matter of handling special cases correctly. The MWE-aware mode is more sensitive. Its high recall suggests that the WordNet-based lexicon and the head-matching rule find most MWEs in the gold data, but precision drops because of lexical ambiguity and conservative choices in the reference set. Some expressions can be read either idiomatically or compositionally depending on context, and this naturally leads to inconsistent decisions.

The character-level tokenizer confirms that local patterns carry a lot of information for Turkish segmentation and that a simple linear model can use these cues effectively. At the same time, it cannot directly incorporate external lexicons or apply user-defined merging rules. In practice, the two tokenizers fill different roles: the rule-based system provides explicit control and MWE handling, while the machine-learning model offers a strong, data-driven default that stays close to UD boundaries.

7 Conclusion and Future Work

This paper has described three core components for Turkish preprocessing: a rule-based stemmer, a rule-based tokenizer with optional MWE merging, and a character-level logistic regression tokenizer. The stemmer reaches 80.2% strict and 93.3% soft accuracy on the BOUN-UD development set. The rule-based tokenizer closely matches UD tokenization (99.73% sentence-level exact match) and achieves 0.82 F1 for MWE merging on a small gold set. The machine-learning tokenizer performs strongly as well, with about 0.99 accuracy and 0.97 macro F1 on the UD test split.

There are several directions for improvement. One option is to add POS tags or lightweight morphological cues to help the stemmer handle ambiguous or heavily derived forms. Another place to refine is the MWE lexicon and the matching rules, especially for increasing precision. For tokenization, it may be useful to explore simple ways of combining character-level predictions with lexicon-based checks, so that boundary decisions and MWE detection can support each other. A longer-term direction is to study how these rule-based and statistical components could be connected to neural sequence models, while still keeping the overall pipeline interpretable for Turkish and other morphologically rich languages.

References

Akın, Ahmet and Mehmet Akın (2007). *Zemberek – Open Source Natural Language Processing for Turkish*. https://github.com/ahmetaa/zemberek-nlp. Accessed: 2025-11-02.

Akın, Ahmet A. and Mehmet D. Akın (2007). "Zemberek: An Open Source NLP Framework for Turkic Languages". In: *Proceedings of the 7th International Conference on Intelligence and Systems*.

Arslan, Emre and Umut Orhan (2016). "Graph-Based Lemmatization of Turkish Words by Using Morphological Similarity". In: *IEEE 24th Signal Processing and Communication Application Conference (SIU)*. IEEE, pp. 1421–1424. DOI: 10.1109/SIU.2016.7496012.

Erjavec, Tomaž, Nikola Ljubešić, Yves Scherrer, Umut Sulubacak, et al. (2022a). *Kenet – Universal Dependencies Turkish Treebank*. https://github.com/UniversalDependencies/UD_Turkish-Kenet. Version 2.12, Accessed: 2025-11-02.

– (2022b). "The Kenet Treebank of Turkish within the Universal Dependencies Framework". In: *Proceedings of the 20th Workshop on Treebanks and Linguistic Theories (TLT 2022)*. Dublin, Ireland: Association for Computational Linguistics, pp. 77–87. URL: https://aclanthology.org/2022.tlt-1.8.

Erjavec, Tomaž, Nikola Ljubešić, Yves Scherrer, and Francis Tyers (2022). *Kenet Turkish WordNet XML Files*. https://github.com/clarin-eric/kenet-wordnet. Accessed: 2025-11-02.

Eryiğit, Gülşen et al. (2015). "Annotation and Extraction of Multiword Expressions in Turkish Treebanks". In: *Proceedings of the 10th Workshop on Multiword Expressions (MWE 2015)*. Denver, Colorado: Association for Computational Linguistics, pp. 70–76. URL: https://aclanthology.org/W15-0912.

Nivre, Joakim et al. (2016). "Universal dependencies v1: A multilingual treebank collection". In: *Proceedings of the Tenth International Conference on Language Resources and Evaluation (LREC'16)*, pp. 1659–1666.

Porter, Martin (2001). *Snowball: A Language for Stemming Algorithms*. https://snowballstem.org/algorithms/turkish/stemmer.html. Accessed: 2025-11-02.

Türk, Uğur et al. (2020). "Resources for Turkish Dependency Parsing: Introducing the BOUN Treebank and the BoAT Annotation Tool". In: *arXiv preprint arXiv:2002.10416*.

Universal Dependencies Consortium (2023). *Universal Dependencies Guidelines, v2*. https://universaldependencies.org/guidelines.html. Accessed: 2025-11-02.